

```
--- djangoapi\scripts\crop\DjangoModels\plantas\plantasDJ.txt ---
```

```
from crop.models import Plantas
from scripts.crop.DjangoModels.tablasDJ import TablasDJ
from django.contrib.gis.geos import GEOSGeometry

class PlantasDJ(TablasDJ):
    def __init__(self):
        super().__init__(Plantas)

    def insert(self, d: dict):
        """Inserta una nueva planta validando que no coincida con otra existente."""
        # 1. Preparación de geometría con Snapping
        wkb = self._get_snapped_wkb(d['geom'])
        g = GEOSGeometry(wkb, srid=self.srid)

        if not g.valid:
            return {'ok': False, 'message': 'Geometría de punto inválida'}

        # 2. Validación T***** (Evita dos puntos en la misma coordenada exacta)
        if self._check_interior_intersection(wkb):
            return {'ok': False, 'message': 'Ya existe una planta en esta ubicación exacta'}

        # 3. Guardar en la base de datos
        d['geom'] = g
        obj = self.model(**d)
        obj.save()
        return {'ok': True, 'id': obj.id}

    def update(self, d: dict):
        """Actualiza los datos o ubicación de una planta."""
        try:
            # 1. Recuperar registro
            obj = self.model.objects.get(id=d['id'])

            # 2. Procesar nueva geometría
            wkb = self._get_snapped_wkb(d['geom'])
            g = GEOSGeometry(wkb, srid=self.srid)

            if not g.valid:
                return {'ok': False, 'message': 'Geometría inválida'}

            # 3. Validar colisión excluyendo el ID actual
            if self._check_interior_intersection(wkb, exclude_id=d['id']):
                return {'ok': False, 'message': 'La nueva ubicación ya está ocupada por otra planta'}

            # 4. Actualizar campos
            obj.variedad = d.get('variedad', obj.variedad)
            obj.estado_salud = d.get('estado_salud', obj.estado_salud)
            obj.fecha_cosecha_est = d.get('fecha_cosecha_est', obj.fecha_cosecha_est)
            obj.geom = g
```

```
        obj.save()
        return {'ok': True, 'message': f'Planta {d["id"]} actualizada
correctamente'}

except self.model.DoesNotExist:
    return {'ok': False, 'message': 'ID de planta no encontrado'}
```